Preguntas de la evaluación, explicadas fácil

Las mismas preguntas que te pueden hacer (de teoría y técnicas), pero respondidas con palabras simples y ejemplos de todos los días. Cada respuesta tiene tres partes:

- **En simple:** lo que tenés que poder decir con tus palabras.
- **Ejemplo:** una comparación para entenderlo.
- **En tu sistema:** cómo se ve en tu proyecto, para señalarlo en la pantalla o en el código.

Un truco para la exposición: no repitas términos técnicos sin explicarlos. Si decís "embedding", agregá enseguida "que es una lista de números que representa el significado".

Bloque 1 — Qué es un RAG

¿Qué es un RAG?

En simple: es un sistema que, antes de contestar, **busca en tus propios documentos** y le pasa lo que encontró a la IA para que conteste con eso. La IA no se acuerda de nada tuyo: lo lee en el momento.

Ejemplo: un examen a libro abierto. Si te preguntan algo, buscás la página, la leés y recién ahí respondés. No lo contestás de memoria.

En tu sistema: el cliente pregunta, Pancho busca en los 3 documentos de PROSEBA, y Gemini escribe la respuesta usando solo esos fragmentos.

Significado de la sigla: *Retrieval-Augmented Generation* = "generación aumentada con recuperación". Recuperar (buscar) + aumentar (pegar lo encontrado) + generar (escribir la respuesta).

¿Por qué no le pasás todos los documentos enteros a la IA en cada pregunta?

En simple: por tres motivos. 1) A veces no entran, porque la IA tiene un límite de lo que puede leer de una vez. 2) Cuesta más y tarda más, porque se paga por cada palabra que se manda. 3) Aunque entre todo, la IA se pierde con textos largos y presta menos atención a lo que está en el medio.

Ejemplo: si le preguntás a un empleado "¿cuánto es la seña?", no le das el manual de 300 páginas: le señalás la página correcta.

En tu sistema: mando solo los **3 fragmentos** más parecidos a la pregunta, no los 26.

¿Por qué la IA "inventa" (alucina) cuando no tiene el dato?

En simple: la IA no tiene una agenda con datos guardados. Aprendió patrones del lenguaje y arma frases que suenan bien. Cuando no sabe algo, igual arma una frase convincente, y la dice con la misma

seguridad que si fuera cierta. No tiene un paso interno que chequee si es verdad.

Ejemplo: alguien que no sabe la respuesta pero habla con tanta seguridad que parece que sí sabe.

En tu sistema: por eso no le pedís que recuerde nada: le das los fragmentos y le ordenás que use solo eso.

Bloque 2 — Troceado (cortar los documentos)

¿Por qué cortás los documentos en fragmentos?

En simple: porque la búsqueda necesita pedazos del tamaño justo: lo bastante chicos para ser específicos, lo bastante grandes para entenderse solos. Si buscás en el documento entero, mezcla financiación, garantía y catálogo, y no se parece del todo a ninguna pregunta.

Ejemplo: si buscás una receta, no querés el libro entero: querés la página de esa receta.

En tu sistema: tus 3 documentos se cortan en **26 fragmentos**.

¿Cómo cortás tu sistema, y por qué así?

En simple: corta **cada vez que aparece un título** (`#`, `##` o `###`). Cada sección es un fragmento. Lo elegiste porque tus documentos ya tienen secciones claras, y así cada pedazo es una idea completa y se puede citar con su título.

Ejemplo: cortar un diario por notas (una nota, un fragmento), en vez de cada 800 letras (que cortaría una nota por la mitad).

En tu sistema: `trocear.mjs` con la "marca" `/^#{1,3} .+/`. Cada modelo del catálogo es su propio fragmento.

¿Qué pasa si una sección es demasiado larga?

En simple: si pasa de 2000 caracteres, se corta entre párrafos (nunca a la mitad de uno) y se repite el título en cada pedazo. Lo que se pierde es el hilo: el segundo pedazo no "sabe" qué decía el primero.

En tu sistema: nunca pasa, porque el fragmento más largo tiene 729 caracteres. El mecanismo está por si el corpus crece.

¿Qué es el solapamiento y por qué no lo usás?

En simple: es repetir un poquito del final de un pedazo al principio del siguiente, para que una idea cortada justo en el borde quede completa en alguno. No lo necesitás porque cortás en los bordes naturales (entre secciones).

Ejemplo: cuando una serie arranca el capítulo con "anteriormente..." para que no pierdas el hilo.

¿Qué harías si tus documentos no tuvieran títulos?

En simple: cortarías cada 800 caracteres, con 150 repetidos entre pedazo y pedazo. Es el plan B que ya está programado. El costo es que los fragmentos quedan sin nombre de sección, y se citarían como "fragmento N".

En tu sistema: se activa poniendo `MARCA = null` en `marca.mjs`.

Bloque 3 — Embeddings y similitud

¿Qué es un embedding?

En simple: una **lista de números que representa el significado de un texto**. En tu sistema son 1024 números por texto.

Ejemplo: las coordenadas de un punto en un mapa. Con dos números (latitud y longitud) ubicás un lugar; con 1024 números "ubicás" un significado. Dos textos que quieren decir lo mismo quedan cerca en el mapa, aunque no compartan ni una palabra.

Por qué sirve: "¿cuánto necesito para empezar a pagar?" y "anticipo mínimo" no tienen palabras en común, pero significan lo mismo. Una búsqueda por palabras exactas falla acá; los embeddings no.

¿Cómo y dónde se calculan los embeddings en tu sistema?

En simple: con un modelo llamado **bge-m3**, que corre en tu propia compu a través de una app llamada **Ollama**. Ningún dato sale de tu máquina para este paso.

En tu sistema: función `embeddingsEnLote()` en `api.mjs`.

¿Por qué los embeddings corren en tu compu y la respuesta en la nube?

En simple: son trabajos muy distintos. Calcular un embedding es **leer** un texto una sola vez. Escribir una respuesta es **crear** texto palabra por palabra, y cada palabra requiere pasar por todo el modelo: eso necesita una máquina potente. Además, indexar hace muchísimos pedidos, y así no gastás el cupo gratuito de Google.

Ejemplo: leer una nota lo hace cualquiera; escribir un libro entero necesita mucho más esfuerzo.

¿Qué mide la similitud coseno?

En simple: mide **hacia dónde apuntan** dos listas de números. Si apuntan para el mismo lado, da 1. Si no tienen nada que ver (perpendiculares), da 0. No importa qué tan "largas" sean, sino la dirección, porque la dirección es la que representa el significado.

Ejemplo: dos personas señalando con el dedo. Si señalan la misma dirección, coinciden (1). Si una señala al norte y otra al este, no tienen relación (0).

Dato técnico: pgvector usa la **distancia** coseno, que es `1 - similitud`. El operador es `<=>`.

¿Por qué no se pueden comparar embeddings de dos modelos distintos?

En simple: cada modelo dibuja su propio mapa. Aunque los dos usen la misma cantidad de números, un punto en un mapa no significa lo mismo en el otro.

Ejemplo: las coordenadas (3, 5) en el mapa de Buenos Aires y en el mapa de Madrid apuntan a lugares totalmente distintos.

En tu sistema: si cambiás de modelo sin volver a indexar, o da error (si la cantidad de números no coincide) o, peor, anda sin error pero los resultados no significan nada. Por eso hay que reindexar y recalibrar el umbral.

Bloque 4 — Base de datos, búsqueda y pgvector

¿Por qué pasaste de un archivo JSON a una base de datos?

En simple: el archivo hay que leerlo entero en cada pregunta aunque uses 3 fragmentos, lo puede usar un solo programa a la vez, y la búsqueda la hacés comparando contra todo. Una base de datos guarda todo de forma permanente, atiende a varios a la vez y tiene un índice para buscar rápido.

Ejemplo: buscar un teléfono en una libreta de papel mezclada (el archivo) contra buscarlo en una agenda ordenada alfabéticamente (la base).

¿Qué es pgvector?

En simple: una **extensión de Postgres** que le enseña a guardar listas de números y a buscar las más cercanas. Le agrega el tipo de columna `VECTOR`, el operador `<=>` y el índice HNSW.

Por qué conviene: en la misma fila guardás el texto, el archivo, la sección y el vector.

¿Dónde está la consulta que busca? (`buscar.mjs`)

```
SELECT id, fuente, seccion, posicion, texto,
       1 - (embedding <=> $1::vector) AS similitud
  FROM fragmentos
 ORDER BY embedding <=> $1::vector
 LIMIT $2
```

En simple: "traeme los fragmentos, ordenados por qué tan cerca están de la pregunta, y quedate con los primeros 3 (`LIMIT`)".

Detalle que preguntan mucho: ordenás por la **distancia** y no por la similitud. Dan el mismo orden, pero solo con la distancia Postgres puede usar el índice rápido (HNSW). La similitud la calculás solo para mostrarla y compararla con el umbral.

¿Por qué se usan `$1`, `$2` en vez de pegar los valores en la consulta?

En simple: para evitar la **inyección SQL**. Si pegaras el texto directo, alguien podría escribir una pregunta que incluya órdenes y hacer que la base las ejecute (por ejemplo, borrar una tabla). Con `$1` los datos viajan **separados** de la orden, y la base nunca los confunde.

Ejemplo: en un formulario, escribir tu nombre en el casillero "nombre" es distinto a que el casillero ejecute lo que escribas como una instrucción.

¿Cómo encuentra HNSW los más cercanos sin mirarlos a todos?

En simple: arma un **mapa con varios pisos**. Arriba hay pocos puntos con conexiones largas (para moverse rápido); abajo están todos, con conexiones cortas (para afinar). La búsqueda arranca arriba, va saltando hacia el vecino más cercano a la pregunta y va bajando de piso.

Ejemplo: buscar una dirección: primero te movés por avenidas grandes (el piso de arriba) y después entrás en las calles chicas (el piso de abajo).

Por qué es "aproximado": puede quedarse en un lugar cercano que no es el más cercano de todos. Cambia un poco de exactitud por muchísima velocidad. Con 26 fragmentos no se nota; importa con miles o millones.

¿Qué pasa si corrés `indexar.mjs` dos veces?

En simple: un `INSERT` **agrega**, no reemplaza. Si no hicieras nada, cada fragmento quedaría duplicado (52 en vez de 26), y el modelo recibiría el mismo texto repetido.

Cómo lo resolviste: `indexar.mjs` empieza con `TRUNCATE TABLE fragmentos RESTART IDENTITY`, que **vacía la tabla** antes de cargar. Siempre quedan 26.

Ejemplo: anotar la lista del súper dos veces en la misma hoja: queda todo duplicado, salvo que borres la hoja antes.

Bloque 5 — El umbral

¿Qué es el umbral?

En simple: es la **nota mínima de parecido** para que un fragmento valga la pena. Si el mejor fragmento no llega, el sistema dice "no lo tengo" **sin llamar a la IA**.

Ejemplo: la nota mínima para aprobar. Debajo de 0,50, el fragmento "no aprueba".

Beneficios: es la defensa más barata contra inventar, y además ahorra la llamada a Google.

¿Cómo elegiste el 0,50? ¿Por qué no un número cualquiera?

En simple: lo **mediste**. Probaste 16 preguntas (11 que están en los documentos y 5 que no) y anotaste la similitud de cada una. Las que están dieron entre **0,505 y 0,786**. Las que no están, entre **0,247 y 0,693**. Elegiste 0,50, justo debajo de la más baja de las que sí están.

Ejemplo: como fijar la nota de corte de un examen mirando cómo les fue a los alumnos, en vez de inventarla.

Las dos nubes de resultados se pisan. ¿Eso no es un problema?

En simple: es normal. **No existe un umbral perfecto.** Cualquier corte puede equivocarse de dos maneras: - **Falso negativo:** una pregunta que sí está queda debajo y el sistema dice "no lo tengo" sin motivo. - **Falso positivo:** una pregunta que no está pasa y llega a la IA.

Si subís el umbral, bajás un error y subís el otro.

Ejemplo: un detector de humo muy sensible suena cuando tostás pan (falsa alarma); uno poco sensible no suena en un incendio real (peor). Hay que decidir cuál error aceptás.

¿Qué error elegiste cometer?

En simple: el **falso positivo**: dejar pasar una pregunta que no está. Porque rechazar una pregunta válida le niega información real al cliente y no se arregla; en cambio, una que se cuela todavía la puede frenar el prompt.

Ejemplo real de tu sistema: la pregunta del **service de la Vela 1.6** pasó el umbral (0,693) aunque el precio no está. El prompt hizo que el modelo no inventara.

¿Por qué el umbral está en el `.env` y no en el código?

En simple: porque **depende del modelo de embeddings y de los documentos**. No sirve de un corpus a otro. De hecho, el umbral del tema anterior (club) era 0,58 y con la concesionaria hubo que bajarlo a 0,50: con 0,58 se habría rechazado la pregunta de los caballos de fuerza de la Travesía, que sí está.

Bloque 6 — El prompt y las capas contra la invención

¿Cómo está armado tu prompt?

En simple: es el mensaje completo que se le manda a la IA, con cinco partes: 1. **Rol:** "Sos Pancho, asesor de ventas de PROSEBA automotores". 2. **Contexto:** los fragmentos encontrados, y nada más. 3. **Tarea:** responder la pregunta con eso. 4. **Restricciones:** no usar conocimiento propio; si no alcanza, decir

exactamente "No lo encuentro en los documentos." 5. **Formato:** respuesta corta (2 oraciones) y un renglón "Fuente:" con archivo y sección.

En tu sistema: la plantilla es `prompt.txt` y la completa `armarPrompt()` en `rag.mjs`, reemplazando `{{fragmentos}}` y `{{pregunta}}`.

¿Por qué exigís una frase exacta para "no sé"?

En simple: para que tu programa pueda **reconocerla**. Si la IA dijera algo distinto cada vez ("no tengo idea", "no consta", "mmm no sé"), el programa no podría saber si encontró o no la respuesta.

¿Por qué exigís citar la fuente?

En simple: para poder **verificar**. Una respuesta sin fuente no se puede comprobar. En tu pantalla, cada respuesta muestra de qué sección salió y, con "Ver los documentos consultados", el texto original.

Ejemplo: en un trabajo escolar, la bibliografía: te permite chequear que lo que dice es cierto.

¿Quién evita que el sistema invente: la búsqueda, el umbral o el prompt?

En simple: los tres, en capas. Ninguno es perfecto solo. 1. **Umbral:** si nada se parece, ni llama a la IA (ej.: "¿quién ganó el mundial?"). 2. **Búsqueda:** limita lo que la IA puede ver a 3 fragmentos. 3. **Prompt:** le ordena usar solo eso y decir "no lo encuentro" si no alcanza.

Ejemplo: la seguridad de un edificio: portería, cerradura y alarma. Lo que se le escapa a una lo frena la siguiente.

Caso real: con el service de la Vela 1.6, la búsqueda trajo fragmentos sin el precio exacto, y fue el prompt el que evitó el precio inventado.

Bloque 7 — Decisiones técnicas y seguridad

¿Dónde está la clave de Google y por qué no puede estar en el navegador?

En simple: está en el archivo `.env`, del lado del **servidor**. Todo lo que llega al navegador se puede leer con F12 (herramientas de desarrollo), clave incluida. Por eso el navegador solo manda la pregunta, y el servidor, que es el único que conoce la clave, habla con Google.

Ejemplo: el cajero del banco no te da la llave de la bóveda: vos pedís, y él va a buscar.

Extra: `.env` está en `.gitignore`, así que no se sube a GitHub.

¿Qué corre en Docker y por qué?

En simple: corre **Postgres con pgvector**, en un contenedor llamado `rag-db`. Un contenedor es una "cajita" aislada que trae todo el programa ya instalado y configurado. Evita instalar Postgres a mano. Los datos se guardan en un volumen (`rag-datos`), así que no se pierden si apagás el contenedor.

Ejemplo: como un electrodoméstico que viene listo para enchufar, en vez de armarlo pieza por pieza.

¿Qué corre en tu compu y qué en la nube?

- **Tu compu:** la base (Docker), el modelo de embeddings (Ollama) y el servidor Node.
- **La nube:** solo Gemini, que escribe la respuesta.

¿Cómo cambiaste de tema (del club a la concesionaria)? ¿Qué tocaste y qué no?

En simple: 1) Reemplazaste los documentos de `corpus/`. 2) No tocaste `marca.mjs`, porque los documentos nuevos también tienen títulos. 3) Adaptaste el primer párrafo de `prompt.txt` (quién responde y de qué), sin tocar las reglas. 4) Escribiste preguntas nuevas, reindexaste y **recalibraste el umbral** (de 0,58 a 0,50).

Lo que no se tocó: el "motor" (`api.mjs`, `buscar.mjs`, `rag.mjs`, `db.mjs`, `trocear.mjs`). Eso demuestra que el sistema sirve para cualquier tema.

¿Qué le falta a tu sistema para usarse en serio?

1. **Reranking:** un segundo modelo más preciso que **reordene** los resultados mirando pregunta y fragmento juntos. Habría distinguido mejor "precio del auto" de "precio del service".
2. **Búsqueda híbrida:** combinar significado con **palabras exactas** (BM25). Sirve para nombres de modelos y datos puntuales.
3. **Actualización incremental:** reindexar solo lo que cambió, no todo.
4. **Medición formal:** probar con cientos de preguntas y medir el porcentaje de aciertos (vos mediste 16 a mano).

Preguntas sobre lo que se ve en la demo

¿Qué significa la barra con la raya roja?

En simple: la barra muestra la **similitud** del fragmento (cuánto se parece a la pregunta), y la raya roja marca el **umbral**. Si la barra llega a la raya o la pasa, el fragmento se usa; si no, aparece como "descartado por el umbral".

¿Qué es "el documento abierto en la sección que vas a mostrar como fuente"?

En simple: cuando mostrás una respuesta que funciona, el docente puede preguntarte "¿cómo sé que eso es verdad?". La prueba es abrir el archivo de verdad (por ejemplo `financiacion.md`), ir a la sección

que cita Pancho ("Planes de financiación propia") y mostrar que ahí dice lo mismo (anticipo mínimo del 30%).

¿Por qué una pregunta que no está pasa el umbral (como la del service de la Vela)?

En simple: porque la similitud mide si el **tema** se parece, no si el **dato exacto** está. La pregunta habla de la Vela y de service, y los dos temas existen en los documentos; lo que falta es el precio. La búsqueda no puede notar esa diferencia fina, y por eso existe el prompt como segunda defensa.

¿Por qué la respuesta cambia un poco si repetís la pregunta?

En simple: el modelo de IA no es 100% determinista: puede redactar distinto cada vez. Lo que **no** cambia son los fragmentos encontrados ni las similitudes, porque eso lo hace la base de datos.

¿Qué pasa si no hay internet o Google falla?

En simple: la búsqueda sigue andando (es local), pero no se puede generar la respuesta. El chat muestra un error con el botón "Reintentar". Si el error es de cupo (429), se puede cambiar el modelo en `.env`.

Glosario para repasar 5 minutos antes

Palabra	En simple
RAG	Buscar en tus documentos antes de que la IA conteste, para que no invente.
Fragmento	Un pedazo de un documento (en tu caso, una sección con título).
Embedding	Lista de números que representa el significado de un texto.
Similitud coseno	Qué tan parecidos son dos significados: 1 es igual, 0 no tienen relación.
Umbral	La nota mínima de parecido. Debajo, el sistema dice "no lo tengo".
Falso positivo / negativo	Dejar pasar algo que no servía / rechazar algo que sí servía.
pgvector	Extensión que le enseña a Postgres a guardar y buscar vectores.
HNSW	Índice que busca saltando entre vecinos en un mapa de varios pisos. Rápido pero aproximado.
Prompt	El mensaje completo que se le manda a la IA: rol, fragmentos, pregunta y reglas.
Alucinación	Respuesta falsa dicha con total seguridad.
Inyección SQL	Ataque que mete órdenes dentro de un dato. Se evita con `$1, $2`.
Docker / contenedor	Una "cajita" que trae un programa ya instalado y listo para usar.
Ollama	App que corre modelos de IA en tu propia compu (acá, para los embeddings).
Gemini	El modelo de Google que escribe la respuesta final (corre en la nube).